

Lalit Devidas Pabil.

Roll No. 106

Div - B. Class - IV

Subject - python

Score

April - 2023

Question Paper - 2023

Python

Q.1 Attempt any eight of the following.

a) What are Special Operators in python.

→ Special Operators in python, often referred to as magic methods or dunder methods (short for 'double underscore'), allow the Customization of behavior for built-in Operations. These methods are defined within Classes and are recognizable by their double underscore before and after their Names.

- Ex.
- i) `-- init --` Initializes a new instance of class.
 - ii) `-- str --` Returns a string repr^s of an object.
 - iii) `-- add (self, other) :` Implements the addition Operator
 - iv) `-- eq -- (self, other) :` Implements equality Comparison.

b) State any four time module.

→ Python has a module named time to handle time-related tasks. To use functions defined in the module, to Import the module first like `import time`.

- i) `time()`
- ii) `gmtime (sec) :`
- iii) `asctime ("time")`
- iv) `ctime (sec)`
- v) `sleep (sec) :`
- vi) `tzset()`
- vii) `clock () :`

c) Difference between python list and Numpy array.

Python List	Numpy array
1) The python core library provided lists.	1) Numpy is the Core library for Scientific Computing in python.
2) Python list requires more memory.	2) Numpy arrays requires less memory consumption and have better runtime behavior.
3) python list operation takes more time and less convenient.	3) Numpy is faster and more convenient than a list.
4) Slices of python list are shallow copies.	4) All arrays generated by Numpy basic slicing are always views of the original array.

d) What is Class Variable?

Class or Static variables are shared by all objects. Instance or Non Static variables are different for different objects (every object has a copy of it). Class variables are defined within the class construction. Because they are owned by the class itself. Class variables are shared by all instances of the class.

Syntax: `class classname (object):`
`self . Instance - variable = value // value specific`

Class - Variable = Value # Value Shared across all Class.

→ accessing instance Variable

Class - Instance = Class Name ()

Class - Instance . instance - Variable

→ accessing Class Variable

Class Name . Class - Variable

e) List Out Geometry management methods.

→

1) The pack(): This method manages the geometry of widgets in blocks.

2) The grid(): The method organizes widgets in blocks.

3) The place(): The method organizes the widgets to place them in a specific.

f) Define term Bind method.

→

The term 'bind method' typically refers to the concept of associating (or binding) a method with a particular instance of a class. When a method is bound to an instance, it means that the method has access to that instance's data (its attributes) and can modify or utilize it as needed.

1) Instance Method.

2) Bound Method.

81. What is Seaborn?

→ Seaborn is a data visualization library built on top of Matplotlib and closely integrated with pandas data structures in python.

Using Seaborn we can plot wide varieties.

- 1) Distribution Plots.
- 2) Pie Chart and Bar Chart
- 3) Scatter plots
- 4) Pair Plots
- 5) Heat maps.

h1. Write any two common exceptions in python.

→

- 1) ArithmeticError - Base class for all errors that occur for numeric calculation.
- 2) Exception - Base class for all exception.

i) What are advantages of pandas.

→

- 1) An Extensive set of features.
- 2) Less Writing, more work done.
- 3) Excellent data representation.
- 4) Make the data flexible and customizable.

j) How to create class and object in python.

python classes are data structure used to define objects. They contain data values and define behavioral characteristics. In python, you define a class by using the key class. Or a class is like an object constructor or a blueprint for creating objects. The class statement creates a new class definition. The name of the class immediately follows the keyword class followed by a colon.

Syntax:-

```
class ClassName:
```

```
<Statement - 1>
```

```
.
```

```
...
```

```
...
```

```
<Statement - N>
```

Q.2. Attempt any four of the following.

1) Explain math and cmath module in detail.

→ Python provides two mathematical modules namely math and cmath. The math module gives us access to hyperbolic, trigonometric, and logarithmic functions for real numbers and cmath allows us to work with mathematical functions for complex numbers.

Ex- for math module.

```
>>> import math
>>> math.ceil(1.001)
```

2

```
>>> from math import *
>>> ceil(1.001)
```

2

```
>>> floor(1.001)
```

1

```
>>> factorial(5)
```

120

```
>>> trunc(1.115)
```

1

```
>>> sin(90)
```

0.8959966686005579

```
>>> cos(90)
```

-0.9524129804151565

```
>>> exp(5)
```

148.4131591025766

```
>>> log(16)
```

2.772588722239781

```
>>> log(16, 2)
```

4.0

```
>>> log(16, 10)
```

1.2041199826559246

```
>>> pow(144, 0.5)
```

12.0

```
>>> sqrt(144)
```

12.0

>>>

Ex: for (math module

```
>>> from math import *
>>> C = 2+2j
>>> exp(C)
(-3.074932320639389+6.71884969342825j)
>>> log(C)
(1.5000000000000002+1.1330900354563985j)
>>> sqrt(C)
(1.5557739740500374+0.6455942529055826j)
```

b) Explain different data types in python.

The type of data value that can be stored in an identifier variable is known as its data type. The data type determines how much memory is allocated to store data and what operations can be performed on it.

1) Number:- Numeric data to perform mathematical operations. Numbers can be integers (like 1 and 2), floats (like 1.1 and 1.2), fraction (like 1/2 and 2/3), or even complex numbers.

2) String - Represent text characters, special symbol or alphanumeric data. String is sequence of Unicode character.

3) List - Represent text character sequential data that the programmer wishes to sort, merge etc.

4) Tuple :- Represent Sequential data with a little difference from list.

5) Dictionary :- Represent a Collection of data that associate a Unique Key with each Value.

6) Boolean :- Represent truth Values (true or false)

7) None :- python defines a Special Variable None denoting a "null object".

c) Explain inheritance in brief with Syntax.

→

Inheritance is a feature of Object Oriented Programming. It is used to specify that one class will get most or all of its features from its parent class. It is a very powerful feature which facilitates users to create a new class with a few or more modification to an existing class. The new class child or derived class and the main class from which it inherits the properties is called base class or parent class.

1) Single Inheritance :- In Inheritance, the child class acquires the properties and can access all the data member and functions defined in the parent class. A child class can also provide its specific implementation to the functions of the parent class.

Syntax :- `class DerivedClassName (BaseClassName):`

`<Statement - 1>`

`<Statement - N>`

2) Multiple Inheritance - Inherit from more than one parent class. Such type of inheritance is called Multiple inheritance. In this case all the attributes and method of both the parent class will be available in the child class after inheritance.

Syntax:

Class A: // define your class A.

.....

Class B: // define your class B.

.....

Class C(A,B): // subclass of A and B

.....

3) Multilevel Inheritance - We can inherit a derived class from another derived class, this process is known as Multilevel inheritance.

Syntax:

Class class 1:

Class class 2(class 1):

Class class 3(class 2):

4) Hybrid Inheritance - Inheritance consisting of multiple types of inheritance is called hybrid inheritance.

5) Hierarchical Inheritance - When more than one derived classes are created from a single base this type of inheritance is called hierarchical inheritance.

d1 Explain Various types of exceptional handling in python.

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program instructions.

For handling exception in python, the exception handler block needs to be written which consists of statements that need to be executed according to raised exception. There are three blocks that are used in the exception handling process, namely, try, except and finally.

1) Try and except:- The try block contains code that may raise an exception. The except block catches the exception and handles it. You can catch specific exceptions or all exceptions.

Ex. try:

```
result = 10/0 # This will raise ZeroDivisionError
except ZeroDivisionError:
    print ("You can't divide by zero!")
```

2) else:- The else block runs if no exceptions are raised in the try block.

try:

```
result = 10/2
```

```
except ZeroDivisionError:
```

```
    print ("You can't divide by zero!")
```

```
else:
```

```
    print ("Division was successful!")
```

8) Finally : The finally block runs no matter what, whether an exception occurred or not. It is typically used for cleanup actions like closing files or releasing resources.

Ex :- try :

result = 10/0

except ZeroDivisionError:

print("you can't divided by zero.")

finally :

print("This will always execute.")

9) Explain principle of Keras.

→

Keras is a high-level neural networks API written in python that simplifies building and training deep learning frameworks like Tensorflow, Theano, and Microsoft Cognitive Toolkit, and it allows developers to quickly prototype, experiment, and deploy neural networks with minimal code.

* Principle *

1) User Friendly API : Keras is designed to make working with deep learning intuitive. The API is simple and consistent so building and training a model is straightforward.

2) Modularity : Keras is built in a modular fashion, meaning each part (neural layers, loss functions, optimizers, etc.) is independent and can be combined to create deep learning models.

3) Support for Multiple Backends: Keras abstracts the backend, allowing the same code to run on TensorFlow, Theano or other deep learning libraries without modification.

Q.3. Attempt any four of the following.

a) What are built in dictionary function in python with example

→ Dictionary functions that allow for the easy manipulation and querying of dictionary data.

1) dict.clear() - Removes all items from the dictionary.

2) dict.copy() - Returns a shallow copy of the dictionary.

Ex. Original dict = {"name": "ABC", "age": 25}
 copied dict = Original dict.copy()
 print(copied dict) ##
 ## output: {'name': 'ABC', 'age': 25}

3) dict.get(key, default) - Returns the value for the specified key if the key is in the dictionary. If not, returns the default value.

Ex. my_dict = {"name": "ABC", "age": 25}
 print(my_dict.get("name"))
 ## output: ABC
 print(my_dict.get("address", "Unknown"))
 ## output: Unknown

4) `dict.items()` - Returns a View object that display a list of dictionary's Key-Value tuple pairs.

Ex. `my_dict = {'name': 'ABC', 'age': 25}`
`print(my_dict.items())`

~~##~~ Output: `dict_items([('name', 'ABC'), ('age', 25)])`

5) `dict.keys()` - Returns a View object displays a list of all the keys in the dictionary.

Ex. `mydict = {'Name': 'PQR', 'age': 25}`
`print(my_dict.keys())`

~~##~~ Output: `dict_keys(['name', 'age'])`

b) Explain Features of Pandas in Python.

→ 1) A fast and efficient Dataframe object for data Manipulation with integrated indexing.

2) Tools for reading and writing data between in-memory data structure and different formats: CSV and text files, MS Excel, SQL database, and the fast HDF5 format.

3) Intelligent data alignment and integrated handling of missing data: Gain automatic label-based alignment in Computations and easily manipulate messy data into an orderly form.

4) Flexible reshaping and pivoting of data sets.

Q) Explain the following with proper Syntax and example entry delete entry . Insert .

Python entry . delete () and entry . Insert () are commonly used with the entry widget in the tkinter library for creating graphical user interfaces (GUIs). They allow the user to manipulate text in an entry field.

Sy. 1) entry . delete (first , last = None)

```
Ex . import tkinter as tk
      root = tk.TK()
      entry = tk.Entry(root)
      entry . pack()
      entry . Insert (0 , "Hello World")
      entry . delete (0 , 5)
      root . mainloop()
```

2) Entry . Insert

Sy. entry . insert (index , string)

```
Ex . import tkinter as tk
      root = tk.TK()
      entry = tk.Entry(root)
      entry . pack()
      entry . insert (0 , "Python")
      entry . insert (tk.END , "Programming")
      root . mainloop()
```

d) Write a Python program to find factors of a given Number.

→

```
def find-factors (num):
    factors = []
    for i in range (1, num + 1):
        if num % i == 0:
            factors.append (i)
    return factors
```

```
number = int (input ("Enter a Number :"))
factors = find-factors (number)
print ("The Factors of {number} are : {factors}")
```

e) Write a python script to generate Fibonacci terms using generator function.

→

```
def fibonacci-generator ():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
fib-gen = fibonacci-generator ()
for _ in range (10):
    print (next (fib-gen))
```

Q. 4) Attempt any four of the following.

Q How to define function in python? Explain with suitable example.

→ 1) Function block begin with the keyword `def` followed by the function name and parentheses `()`.

2) The Input parameters or arguments should be placed within these parentheses.

3) The first statement of a function can be an optional statement - the documentation string of the function or docstring or documentation string. It is used to explain in brief, what a function does.

4) The code block within every function starts with a colon and is indented.

5) The return statement is used to exit a function. A return statement with no argument is the same as return `Note.`

Ex:-

```
def my-function():
    print("Hello from a function")

def fun(str)
    "This prints a passed string into this fun"
    print str
    return.
```

Explain EXCEPT Clause with no exception.

EXCEPT Clause with no exception, It can handle all type of exceptions.

Syntax :-

try :

You do your operations here

except :

If there is any exception, then execute this block.

else :

If there is no exception then execute this

Ex.

try :

a = 10 / 0 ;

except :

print "Arithmetic Exception"

else :

print "Successfully Done"

Output :-

Arithmetic Exception

Q1 Explain IS - A relationship and HAS - A relationship with example.

→ The is-a and has-a relationship are used to define how Classes and Objects relate to one another.

1) IS - a Relationship (Inheritance) :- The is-a relationship is established through inheritance. It defines a parent-child (super class - subclass) relationship. The Subclass 'is a' type of the Superclass, meaning the Subclass inherits properties and behaviour (method) of the Superclass.

Ex 1. # SuperClass

class Vehicle:

def __init__(self, brand, year):

self.brand = brand

self.year = year

def start(self):

return f"{self.brand} Vehicle is

Starting.

SubClass

class Car(Vehicle):

def __init__(self, brand, year, doors):

super().__init__(brand, year)

self.doors = doors

def start(self):

return f"{self.brand} Car with {self.doors} doors is Starting."

```

my-car = Car ("Toyota", 2020, 4)
print (my-car.start ())
print (f"Car year: {my-car.year}")

```

c) Has - a Relationship :- The has-a relationship is established through Composition. This means that a Class contains an instance of another class as a member, indicating that 'this object has an instance of that object'.

Ex:

```

class Engine:

```

```

    def __init__(self, horsepower):

```

```

        self.horsepower = horsepower

```

```

class Car:

```

```

    def __init__(self, model, engine):

```

```

        self.model = model

```

```

        self.engine = engine

```

```

engine = Engine (150)

```

```

car = Car ("Toyota", engine)

```

```

print (car.model)

```

```

print (car.engine.horsepower)

```

d) Write a python program to check if a given key already exists in a dictionary. If key exists replace with

```

def replace-key-if-exists (dictionary, old-key, new-key,
new-value):

```

Old-key in dictionary :

```
dictionary.pop(old-key)
```

```
dictionary[new-key] = new-value
```

```
print(f"key '{old-key}' found and replaced with  
{new-key} : {new-value}")
```

else :

```
print(f"Key '{old-key}' not found in the dictionary")
```

```
my-dict = {'a': 1, 'b': 2, 'c': 3}
```

```
print("Original dictionary", my-dict)
```

```
replace-key-if-exists(my-dict, 'b', 'new-key', 99)
```

```
print("updated dictionary:", my-dict)
```

Ex: Write a python program to Swap the Value of two Variables.

→

```
def swap-values(a, b)
```

```
print("Before Swapping : a = ", a, "b = ", b)
```

```
a, b = b, a
```

```
print("After Swapping : a = ", a, "b = ", b)
```

```
x = 5
```

```
y = 10
```

```
swap-values(x, y)
```

Q.5. Write short notes on.

1) Slicing Dictionaries :- Slicing generally refers to extracting a portion of a sequence like lists or strings. However, dictionaries are unordered collections of key-value pairs, so they cannot be sliced like sequences. But you can extract specific key-value pairs based on certain criteria or keys. This can be done using dictionary comprehension, dict() functions, or manually iterating over keys.

Ex:-

```
my-dict = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
sliced-dict = {k: my-dict[k] for k in ['a', 'c']}
print(sliced-dict)
```

2) Data visualization :- ~~Slicing~~ generally refers to Data visualization refers to the graphical representation of data to help users understand trends, patterns, and outliers within large data sets. Using visual tools like charts, graphs, and maps, data becomes more intuitive and accessible for decision-making. Common Python libraries for data visualization.

1) Matplotlib :- Basic plotting like line graphs, bar charts, and histograms.

2) Seaborn :- Statistical visualization built on top of matplotlib, offering more attractive and easy-to-use plots.

3) Plotly :- Interactive plots and dashboards.

3) Custom Exception :- A custom exception in python is a user defined exception that is created by inheriting from python's built-in Exception class. It allows developers to handle specific error scenarios that are not covered by built-in exceptions. Custom exceptions help make the error message more descriptive and relevant to the application's logic.

Ex. Class InvalidAgeError (Exception):

```
def __init__(self, message = "Age must be between 18 and 60"):
```

```
    self.message = message
```

```
    super().__init__(self.message)
```

```
def check_age(age):
```

```
    if not 18 <= age <= 60:
```

```
        raise InvalidAgeError()
```

```
try:
```

```
    check_age(15)
```

```
except InvalidAgeError as e:
```

```
    print(e)
```

25/12/24
Key pad